

Chapter 1 Review

Learning Objectives

- Consolidate understanding of functions, arrays, loops, recursion, and algorithms
- Solve 综合 problems that combine multiple programming concepts
- Reflect on learning progress and identify areas for improvement

Engage (5-7 minutes)

Over the past eleven lessons, you have built a strong foundation in programming—from functions and arrays to recursion and algorithm design. Today is not just a review; it is a chance to see how all these concepts connect and work together to solve real problems. Think of it as assembling a puzzle where each piece you learned fits into a bigger picture.

Explore (10-12 minutes)

Work in pairs to solve this challenge: given an array of student scores, write a program that calculates the average, finds the highest and lowest scores, counts students above average, sorts scores in descending order, and displays results with proper formatting. Identify which concepts from this chapter you need: functions for each calculation, arrays for storage, loops for processing, and string operations for output formatting.

Explain (10-12 minutes)

This chapter covered the building blocks of programming. Functions provide modularity and reusability. Arrays and their operations enable efficient data storage and manipulation. Nested loops handle multi-dimensional problems. Recursion offers elegant solutions for problems with self-similar structure. Algorithm design and problem-solving strategies give you a systematic approach to any challenge. Debugging skills ensure your code works correctly. Together, these concepts form the foundation for all advanced programming topics.

Elaborate (8-10 minutes)

Complete this comprehensive review project: build a grade management system that stores student names and scores in arrays, uses functions to calculate statistics (average, median, standard deviation), implements search and sort algorithms, handles user input with validation, and generates formatted reports. Include at least five functions, two types of loops, and proper error handling. Test with 20 students and verify all calculations manually.

Evaluate (5-8 minutes)

Take a self-assessment quiz covering all chapter topics. Then write a reflection: which concept was most challenging and why, which was most interesting, how you would explain recursion to a younger student, and what programming goal you want to achieve next. Submit your project code with documentation and your reflection.